LAPORAN PRAKTIKUM PEMROGRAMAN BERORIENTASI OBJEK

Relasi Antar Objek

Disusun Oleh :

Aliifah Felda Mufarrihati Salwaa

2511531011

Link Repository Gitbuh PBO:

https://github.com/aliifah-felda/PrakPBO_C_2511531011.git

Dosen Pengampu :

Nurfiah, S.ST, M.Kom



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

A. Langkah Kerja Praktikum

1. Membuat Class Diagram Transaksi

1.1. Buat class dengan nama transaksi

```
public class Transaksi {}
```

1.2. Buat atribut dan konstruktor

```
public class Transaksi {
	String idTransaksi;
	String jenis;
	double nominal;
	NumberFormat nf = NumberFormat.getInstance(Locale.GERMAN);

	//construktor
	public Transaksi(String id, String jenis, double nominal) {
		this.idTransaksi = id;
		this.jenis = jenis;
		this.nominal = nominal;
	}
```

Class transaksi memiliki 3 attribute yakni:

- idTransaksi : untuk mendapatkan id dari setiap transaksi yang nantinya akan dihasilkan secara otomatis menggunakan waktu transaksi.
- jenis : untuk menentukan jenis transaksi dimana ada transaksi debit dan kredit.
- nominal : berapa nominal yang digunakan pada setiap transaksi.

1.3. Buat method

```
public void cetakDetail() {
		System.out.println("ID: " + idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp" + nf.format(nominal));
	}
```

Membuat method cetakDetail() berfungsi untuk menampilkan transaksi secara keseluruhan meliputi: idTransaksi, jenis, dan nominal. Ini sangat efektif ketika kita akan melakukan pencetakan transaksi dan mengetahui setiap detail transaksi.

1.4. Kode Program Lengkap Class Transaksi

```
package Pertemuan2;

import java.text.NumberFormat;
import java.util.Locale;
```

```
public class Transaksi {
	String idTransaksi;
	String jenis;
	double nominal;
	NumberFormat nf = NumberFormat.getInstance(Locale.GERMAN);

	//construktor
	public Transaksi(String id, String jenis, double nominal) {
		this.idTransaksi = id;
		this.jenis = jenis;
		this.nominal = nominal;
	}

	public void cetakDetail() {
		System.out.println("ID: " + idTransaksi + " | Jenis: " +
jenis + " | Nominal: Rp" + nf.format(nominal));
	}



}
```

Disini melakukan penambahan number format agar nominal atribut nominal nantinya keluar dengan format number, seperti 80.000 dan bukan 80000.

2. Membuat Relasi pada Rekening.java

```
public class Rekening {			// Membuat class rekening dengan
rekening.java
   String nomorRekening;	// Membuat state/atribut
   String namaPemilik;
   double saldo;
   NumberFormat nf = NumberFormat.getInstance(Locale.GERMAN);		//
Challenge Number format untuk mengubah 50000 menjadi 50.000

   ArrayList<Transaksi> riwayatTransaksi; //membuat array untuk
menyimpan data

   public Rekening(String nomor, String nama, double saldoAwal) {
   // Membuat konstruktor rekening
	this.nomorRekening = nomor;
	this.namaPemilik = nama;
	this.saldo = saldoAwal;

	this.riwayatTransaksi = new ArrayList<>(); //menginstall
arraylist dalam constructor agar tidak NullPointerException

	System.out.println("Rekening atas nama " + namaPemilik + "
berhasil dibuat dengan saldo Rp" + nf.format(saldo));
   }
```

Pada class rekening, kita menambahkan atribut baru yakni arraylist agar sistem dapat mencatat riwayat transaksi yang terjadi. Kemudian disini kita juga melakukan inisialisasi riwayatTransaksi pada konstruktor.

3. Mencatat Riwayat Transaksi

```
public void setorTunai(double nominal) {          // Membuat method setor
tunai
          if (nominal > 0 && nominal <= 500000) {
                saldo += nominal;
                String idTrx = "TRX-S-" + System.currentTimeMillis();
                Transaksi trxBaru = new Transaksi(idTrx, "Kredit",
nominal);
                riwayatTransaksi.add(trxBaru);

                System.out.println("Setor tunai Rp" + nf.format(nominal)
+ " berhasil. Saldo saat ini: Rp"+ nf.format(saldo));
          } else if (nominal <= 0) {
                System.out.println("Gagal: Nominal setor harus lebih
dari 0!");
          } else {
                System.out.println("Gagal: Nominal setor harus kurang
dari Rp500.000!");
          }
    }
```

Disini kita akan membuat bagaimana agar transaksi itu dapat tercatat dengan baik dan mengatur bahwa idTransaksi dari setorTunai harus menggunakan awalan TRX-S dan dapat tampil dengan jenis kredit.

B. Tugas/Latihan Praktikum

1. Integrasi Rekam Jejak Penarikan

```
public void tarikTunai(double nominal) {          // Membuat method tarik
tunai disini adalah tugas pertama
          if (nominal > 0) {
                if (nominal > saldo) {            // Agar saldo tidak
bisa minus
                      System.out.println("Transaksi gagal: Saldo tidak
mencukupi. Saldo anda Rp" + nf.format(saldo));
                } else {
                      saldo -= nominal;
                      String idTrx = "TRX-T-" +
System.currentTimeMillis();
                      Transaksi trxBaru = new Transaksi(idTrx, "Debit",
nominal);
                      riwayatTransaksi.add(trxBaru);
                      System.out.println("Tarik tunai Rp" +
nf.format(nominal) + " berhasil. Saldo saat ini: Rp"+ nf.format(saldo));
                }
```

```
        } else {
                System.out.println("Gagal: Nominal tarik tunai harus
lebih dari 0!");
        }
    }
```

Disini kita akan melakukan hal yang sama atau logika yang sama dengan setorTunai. Perbedaannya, ketika setor tunai, idTransaksi menggunakan TRX-S dan tampil sebagai jenis kredit. Sementara itu jika terjadi tarikTunai, id transaksi menggunakan TRX-T dan akan tampil sebagai jenis debit.

Hal ini juga harus tetap mempertahankan prinsip dimana nominal tariTunai tidak boleh melebihi saldo yang dimiliki.

2. Fitur Cetak Mutasi

```
    public void cetakMutasi() {
        if (riwayatTransaksi.isEmpty()) {
                System.out.println("Belum ada transaksi pada rekening
ini");
        } else {
                System.out.println("--- MUTASI REKENING ---");
                for (Transaksi transaksi : riwayatTransaksi) {
                        transaksi.cetakDetail();
                }
        }
    }
```

Disini kita akan membuat method bernama cetakMutasi() agar dapat menelusuri data di dalam koleksi riwayatTransaksi. Menggunakan for untuk memanggil method cetakDetail dan jika belum ada riwayat tampilkan pesan.

```
System.out.println("6. Cek Mutasi");
```

```
case 6:
        akunAktif.cetakMutasi();
        break;
```

Kemudian kita akan membuka file main dan menambahkan menu baru, menu ke-6 yakni Cek Mutasi. Pada menu ini kita akan memanggil method cetakmutasi() untuk menampilkan mutasi terbaru dari rekening aktif saat ini.

C. Output yang dihasilkan

```
=== SISTEM PERBANKAN MINI ===

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Rekening
6. Cek Mutasi
7. Tampilkan 3 Transaksi Terbaru
0. Keluar
Pilih menu: 1
Masukkan No Rekening: 12
Masukkan Nama Pemilik: ali
Masukkan Saldo Awal: 90000
Rekening atas nama ali berhasil dibuat dengan saldo Rp90.000

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Rekening
6. Cek Mutasi
7. Tampilkan 3 Transaksi Terbaru
0. Keluar
Pilih menu: 2
Masukkan nominal setor: 10000
Setor tunai Rp10.000 berhasil. Saldo saat ini: Rp100.000
```

```
Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Rekening
6. Cek Mutasi
7. Tampilkan 3 Transaksi Terbaru
0. Keluar
Pilih menu: 3
Masukkan nominal tarik tunai: 30000
Tarik tunai Rp30.000 berhasil. Saldo saat ini: Rp70.000

Menu Utama:
1. Buka Rekening Baru
2. Setor Tunai
3. Tarik Tunai
4. Cek Informasi Rekening
5. Ganti Rekening
6. Cek Mutasi
7. Tampilkan 3 Transaksi Terbaru
0. Keluar
Pilih menu: 6
--- MUTASI REKENING ---
ID: TRX-S-1790093598494 | Jenis: Kredit | Nominal: Rp10.000
ID: TRX-T-1790093601992 | Jenis: Debit | Nominal: Rp30.000
```

Disini kita dapat melihat bagaimana program mutasi rekening dapat berjalan dengan baik. Dimana setor tunai dan tarik tunai berjalan dengan benar dan tercatat pada sistem mutasi sehingga mutasi dapat dilhat detailnya mulai dari idTransaksi, jenis, dan nominal. Hal ini tentu akan lebih memudahkan user dalam memanajemen keuangan di bank tersebut karena mutasinya jelas dan tersimpan.